struct vs class en Swift

420-249-AH - Programmation mobile iOS

1. Pourquoi comparer struct et class ?

En Swift, struct et class permettent toutes les deux de regrouper :

- Propriétés
- Méthodes
- Initialisation
- Comportements

Mais elles ne fonctionnent pas de la même manière en mémoire.

La différence essentielle est :

Mot-clé	Comportement
`struct`	Type valeur
`class`	Type référence

2. Définir une struct

Une structure se déclare avec :

```
struct
```

Exemple :

```
struct Appareil {
    var nom: String
    var batterie: Int
}
```

Création :

```
var iphone = Appareil(
    nom: "iPhone",
    batterie: 80
)
```

3. Initialisateur automatique

Pour une structure simple, Swift fournit automatiquement un initialisateur membre par membre.

Avec :

```swift
struct Appareil {
    var nom: String
    var batterie: Int
}
```

on peut directement écrire :

```swift
let appareil = Appareil(
    nom: "iPhone",
    batterie: 80
)
```

Il n'est pas nécessaire d'écrire explicitement init.

4. Une struct peut avoir des méthodes

```swift
struct Appareil {
    var nom: String
    var batterie: Int

    func afficherEtat() {
        print("\(nom) : \(batterie)%")
    }
}
```

Utilisation :

```swift
let appareil = Appareil(
    nom: "iPhone",
    batterie: 80
)
appareil.afficherEtat()
```

5. Modifier une propriété dans une méthode

Une structure est un type valeur.

Une méthode qui modifie une propriété doit être déclarée avec :

```swift
mutating
```

Exemple :

```swift
struct Batterie {
    var niveau: Int

    mutating func consommer(_ quantite: Int) {
        niveau -= quantite
    }
}
```

Utilisation :

```
var batterie = Batterie(niveau: 100)
batterie.consommer(20)
print(batterie.niveau)
```

Résultat :

```
Console
80
```

6. Pourquoi mutating ?

Dans une structure, une méthode normale ne peut pas modifier directement l'état de l'instance.

Ceci est incorrect :

```
struct Batterie {
    var niveau: Int

    func consommer() {
        niveau -= 10
    }
}
```

Il faut écrire :

```
mutating func consommer() {
    niveau -= 10
}
```

7. Une structure déclarée avec let

Avec :

```
let appareil = Appareil(
    nom: "iPhone",
    batterie: 80
)
```

on ne peut pas modifier :

```
appareil.batterie = 50
```

Même si batterie a été déclarée avec var. Pourquoi ? Parce que l'instance complète de la structure est constante.

8. Comparaison avec une classe

Avec une classe :

```
class Appareil {
    var batterie: Int = 80
}
```

Puis :

```
let appareil = Appareil()
```

ceci reste permis :

```
appareil.batterie = 50
```

C'est une différence importante :

Combinaison	Effet
`struct + let`	Aucune propriété modifiable
`class + let`	La référence est constante, mais les propriétés var peuvent changer

9. struct = type valeur

Considérons :

```
struct Position {
    var x: Int
    var y: Int
}
```

Puis :

```
var position1 = Position(x: 10, y: 20)
var position2 = position1
```

position2 reçoit une copie de position1.

10. Modifier la copie

```
position2.x = 100
```

Résultat :

```
position1.x → 10
position2.x → 100
```

Les deux valeurs sont indépendantes.

```
position1
   ↓ copie
position2
```

11. class = type référence

Prenons maintenant :

```
class Position {
    var x: Int
    var y: Int

    init(x: Int, y: Int) {
        self.x = x
        self.y = y
    }
}
```

Puis :

```
let position1 = Position(x: 10, y: 20)
let position2 = position1
```

Ici, aucun nouvel objet n'est créé. Les deux variables référencent la même instance.

12. Modifier une référence

```
position2.x = 100
```

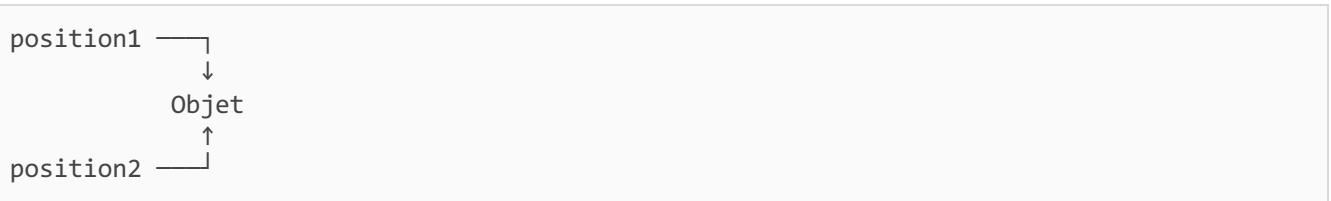
Puis :

```
print(position1.x)
```

Résultat :

```
Console
100
```

Pourquoi ?



Les deux références pointent vers le même objet.

13. Value Type vs Reference Type

struct	class
Type valeur	Type référence
Copie de la valeur	Partage d'une référence

`Pas d'héritage`	Héritage possible
`mutating pour modifier dans une méthode`	Pas de mutating
`Initialisateur automatique fréquent`	init généralement nécessaire
`Pas d'identité d'objet`	Identité d'objet

14. Les types Swift courants sont souvent des structures

Plusieurs types utilisés depuis le début du cours sont des types valeur.

Exemples :

```
Int
Double
Bool
String
Array
Dictionary
Set
```

Ils se comportent donc conceptuellement comme des valeurs copiées.

15. Exemple avec Array

```
var liste1 = ["iPhone", "iPad"]
var liste2 = liste1
liste2.append("Mac")
```

Résultat conceptuel :

```
liste1 → iPhone, iPad
liste2 → iPhone, iPad, Mac
```

Modifier liste2 ne modifie pas liste1.

16. Identité des objets de classe

Deux variables peuvent référencer exactement le même objet.

Swift permet de vérifier cela avec :

```
===
```

Exemple :

```
let appareil1 = Appareil()
let appareil2 = appareil1

if appareil1 === appareil2 {
    print("Même instance")
}
```

Résultat :

```
Console
Même instance
```

17. == et ===

Il ne faut pas confondre :

```
==
===
```

Opérateur	Signification
==	Les valeurs sont-elles équivalentes ?
===	Est-ce exactement la même instance de classe ?

=== est utilisé avec les classes.

18. Quand utiliser une struct ?

Une structure est généralement un bon choix lorsqu'on représente principalement une valeur.

Exemples mobiles :

- Position GPS
- Coordonnées
- Configuration
- Produit simple
- Dimension
- Couleur personnalisée
- Données d'un profil
- État simple

Exemple :

```
struct PositionGPS {
    var latitude: Double
    var longitude: Double
}
```

Deux positions peuvent être copiées indépendamment.

19. Quand utiliser une class ?

Une classe est utile lorsqu'on veut représenter une entité avec une identité et un état partagé.

Exemples :

- Gestionnaire de session

- Connexion réseau
- Service
- Contrôleur
- Objet partagé entre plusieurs composants
- Entité nécessitant de l'héritage

Exemple :

```
class SessionUtilisateur {
    var estConnecte: Bool = false
}
```

Plusieurs parties du programme peuvent travailler avec la même instance.

20. Règle pratique en Swift

En Swift, on privilégie souvent struct lorsque la sémantique de valeur convient.

On choisit class lorsqu'on a réellement besoin :

- D'identité
- De références partagées
- D'héritage
- Ou d'un cycle de vie d'objet spécifique

21. Exemple - Position GPS avec struct

Reprenons PositionGPS pour observer concrètement le comportement de copie :

```
var anciennePosition = PositionGPS(
    latitude: 45.50,
    longitude: -73.56
)

var nouvellePosition = anciennePosition
```

Modification :

```
nouvellePosition.latitude = 46.0
```

L'ancienne position reste inchangée. C'est exactement le comportement attendu pour une valeur représentant des coordonnées.

22. Exemple - Session avec class

```
class Session {
    var utilisateur: String
    var estConnecte: Bool
```

```
    init(utilisateur: String) {
        self.utilisateur = utilisateur
        self.estConnecte = true
    }
}
```

Puis :

```
let sessionPrincipale =
    Session(utilisateur: "Alex")
let sessionCourante =
    sessionPrincipale
```

Si :

```
sessionCourante.estConnecte = false
```

alors :

```
sessionPrincipale.estConnecte
```

vaut également false. Les deux variables utilisent la même session.

23. Exemple guidé - Comparer le comportement

Créer :

```
struct BatterieValeur {
    var niveau: Int
}
```

et :

```
class BatterieReference {
    var niveau: Int

    init(niveau: Int) {
        self.niveau = niveau
    }
}
```

Tester ensuite :

```
var batterie1 =
    BatterieValeur(niveau: 100)
var batterie2 = batterie1
batterie2.niveau = 50
```

Observer les deux valeurs. Puis faire la même expérience avec BatterieReference. Comparer les résultats.

24. Exercice - Profil : struct ou class ?

Créer une structure :

```
struct Profil {
```

avec :

```
nom
age
ville
```

Créer :

```
var profil1 = ...
var profil2 = profil1
```

Modifier la ville de profil2. Afficher les deux profils.

Question : pourquoi profil1 n'est-il pas modifié ?

Créer ensuite une classe :

```
class SessionProfil
```

avec :

```
nomUtilisateur
estConnecte
```

Créer :

```
let session1 = ...
let session2 = session1
```

Modifier :

```
session2.estConnecte
```

Afficher ensuite l'état depuis les deux variables.

Question : pourquoi les deux valeurs ont-elles changé ?

25. À retenir

struct et class peuvent toutes les deux contenir :

- Propriétés
- Méthodes
- Initialisateurs

Mais la différence fondamentale est :

Type	Comportement
`struct`	Value Type — copie indépendante
`class`	Reference Type — instance partagée

Dans une structure, mutating est nécessaire lorsqu'une méthode modifie l'état.

Une classe permet également :

- Héritage
- Identité d'instance
- Références partagées

26. Vision globale

```
struct
  │
  ├── type valeur
  ├── copie
  ├── mutating
  └── pas d'héritage

class
  │
  ├── type référence
  ├── partage d'instance
  ├── identité
  └── héritage
```